

PulseOS

Version 2 — “Afterglow” · September 2026

1. Introduction

Pulse OS is a personal command-centre dashboard that brings the many separate services people check every day into a single, calm, unscrolling interface. Where a typical routine means opening one app for the weather, another for the calendar, another for news, markets, sport, music and travel, Pulse OS gathers these domains into one cohesive surface. The aim of the project is to reduce the friction and noise of jumping between applications by presenting everything the user needs in one glance, in a layout that never requires the page to scroll.

The system is built as a monorepo with a React and Vite frontend and a Node.js and Express backend that acts as a secure gateway to a wide range of third-party providers. The backend follows a clean, layered architecture that separates routing, controllers, services and provider integrations, which keeps each external service isolated and the codebase maintainable. The frontend mirrors this separation with feature components, state hooks and a single API client. Beyond the dashboard itself, the project includes a provider-swappable AI assistant with both typed and real-time spoken interfaces and with read and write access to the user's own data; live news streaming; real-time market and sports data; calendar synchronisation across Google and Apple; live travel data including flight tracking and mapping; on-device music playback control with time-synced lyrics; and a request rate limiting and usage capping system that keeps the application inside free-tier limits.

Version 2, released in September 2026, is a full rebuild of the frontend. The first version organised every screen as a grid of frosted glass cards floating on a fixed gradient, with centred, letter-spaced titles. Version 2 replaces that with a single spatial system shared by every view, a typographic scale, a living background whose colour follows the time of day, and a redesigned navigation bar. It also substantially extends what the product can do: the travel planner became a live, connected feature, the assistant gained real-time voice, and the application launcher was promoted from a modal dialogue to a view of its own. One feature, the Finance overview, was removed.

1.5 Project Function and What It Does

Pulse OS is an all-in-one personal dashboard designed to help a user manage their whole day from one screen. From the user's point of view, the app opens to a personalised home view that greets them by name and immediately shows the weather, a multi-day forecast, their upcoming calendar events, their next work shift and a launchpad of their most-used applications. Instead of checking a weather app, a calendar app and a news app separately, Pulse OS presents these together in one experience.

Beyond the home view, the app supports the user across six further areas: a Launchpad that opens real applications on the machine and searches the web; a Life Hub for schedules, to-dos, projects and daily habits; a Markets & News area with live television news, a stock ticker and configurable news, sport and market feeds; a Music player with an immersive full-screen mode; a Travel planner with live flight, map and currency data; and an AI assistant that sits at the centre of the navigation and can read the user's live data and make changes on their behalf, either typed or spoken aloud. The main function of the project is to make daily life easier to manage by combining information, planning and personal services into a single, unified platform.

2. Tech Stack Used

2.1 Core technologies

- JavaScript (ES Modules)
- React and Vite
- Tailwind CSS
- Node.js and Express.js
- Firebase (Cloud Functions and Hosting ready)
- WebSockets (`ws`) for real-time voice
- hls.js for live television streaming
- MapLibre GL for mapping
- lucide-react for iconography
- Newsreader, Schibsted Grotesk and Vazirmatn typefaces

2.2 External services

Domain	Provider
Weather	OpenWeather
Stocks and cryptocurrency	Finnhub, CoinGecko
News	GNews with RSS feeds; local news resolved by town or county
Sport	Football-Data.org (football), balldontlie (NBA, NFL), Jolpica/Ergast (Formula 1)
Calendar	Google Calendar (OAuth), Apple iCloud (CalDAV), public <code>.ics</code> feeds
Music	Spotify Web API, Web Playback SDK and Connect; LRCLIB for lyrics
Travel	Google Places or OpenStreetMap, AeroDataBox, OpenFreeMap tiles
Web search	Brave, Tavily, or a keyless open-web provider

AI	Google Gemini (default), OpenAI, Anthropic Claude
Voice	Google Gemini Live API

2.3 Why these choices

React and Vite were used to build the frontend because they allow a fast, modern, component-driven interface with a very quick development feedback loop, which suited a project made up of many independent features that each manage their own data. Tailwind CSS was used for styling because it made it possible to build a consistent visual language quickly and without a large custom stylesheet, while still allowing a hand-written layer for the design system described in section 3.3. Lucide-react provided a clean, consistent icon set, hls.js was used to stream live television news directly inside the browser, and MapLibre GL was chosen for mapping because it renders vector tiles from the free OpenFreeMap service without requiring a commercial mapping key.

On the backend, Node.js and Express were used to build a clear API gateway, with each domain given its own route, controller and service so that request handling stays separate from the logic that talks to each external provider. Firebase was chosen as the deployment target because the backend is structured so that the Express application can be wrapped by Cloud Functions without changing any route or service, and Firebase Hosting can serve the built frontend.

A deliberately wide set of third-party APIs was integrated to cover each domain of the dashboard, and in every case a free tier was chosen. For the AI assistant, Google Gemini was selected as the default provider because it offers the most generous free tier and supports function calling, with OpenAI and Anthropic Claude available as drop-in alternatives selectable per request. For real-time voice, the Gemini Live API was used because it handles turn-taking, voice activity detection and interruption server-side, and because it accepts the same function-calling tool definitions already written for the typed assistant.

3. Architecture Used

3.1 Overall shape

Pulse OS follows a feature-based, layered architecture across a single monorepo managed with npm workspaces, with the frontend and backend kept as separate packages. It does not rely on a heavy framework, but responsibilities are clearly separated.

3.2 Backend layering

On the backend, every incoming request passes through a defined chain: a route file maps the URL to a controller, the controller validates input and delegates to a service, and the service coordinates one or more providers that each wrap a single external API. This means, for example, that the weather route never talks to OpenWeather directly — it goes through a weather controller, a weather service, and finally an OpenWeather provider — so any provider can be swapped or extended without touching the rest of the code.

Supporting this are a set of cross-cutting layers. A single configuration module reads every environment variable and exposes a central view of the application's settings, so nothing else in the codebase reads process variables directly. A middleware layer handles CORS, request rate limiting and centralised error handling, turning typed application errors into clean JSON responses. A utilities layer provides shared tools including a time-to-live cache that serves stale data rather than failing when a provider is rate-limited, a persisted token store for OAuth sessions, and a persisted usage meter that enforces hard spend caps on the metered AI providers.

Importantly, the Express application is constructed separately from the call that starts the server, which is what makes the backend ready to run either locally or inside Firebase Cloud Functions with no code changes. The real-time voice WebSocket is attached in the local server bootstrap rather than in the application factory, deliberately, so that the Firebase deployment path remains unaffected by it.

3.3 Frontend layering and the design system

On the frontend, the interface is organised into feature components, each representing a view or a part of one, backed by dedicated state hooks such as those for weather, calendar events, life data, market data, news, travel, lyrics and the Spotify player. All communication with the backend flows through one API client, which is the single place the frontend talks to real integrations.

Version 2 adds a shared spatial system on top of this. Every view is composed of the same three parts, defined once in a single module: a *sky zone* that holds what the view is about, set large and directly on the background; a *horizon* that falls at the same height in every view, so that moving between tabs does not shift the layout; and a *ground*, one full-width surface that runs from the horizon to the bottom of the window and is divided into columns by space and a faint rule rather than broken into floating cards. Only genuinely physical objects — an album sleeve, the live video, the map, an application icon — are still drawn as tiles. A supporting hook measures list rows so that a scrolling column always shows a whole number of them and never ends on a row cut in half.

The background itself is a component. A sky module writes colour values to registered CSS custom properties according to the hour of the day, giving a peach dawn, a clear blue day, an ember dusk and an indigo night with stars; because the properties are registered, a change of phase crossfades rather than snapping. The Music view can temporarily lend the sky the dominant colours of the current album sleeve, so the light behind the interface carries meaning rather than decorating it. Typography is governed by a fixed eight-step scale, with Newsreader used for what is read at a distance and Schibsted Grotesk for what is used up close.

This layered separation on both sides of the application makes each part easy to understand, test and maintain, because every feature has a clear and isolated purpose, and every view is assembled from the same small set of structural pieces.

4. Full Functionality with Technical Implementation

4.1 Shell, navigation and settings

The application is organised around a persistent navigation bar that gives access to seven areas: Home, Launchpad, Life Hub, the AI assistant, Markets & News, Music and Travel. In version 2 this bar is drawn as a hairline running the full width of the foot of the screen with the tabs standing on it and the current tab's label underlined. The assistant is not a button on the line but is rendered as the line itself, taking the form of an ECG beat at the centre, with a light blip that travels along the rule and eases through the beat. The line and the beat are generated as a single measured SVG path so that they cannot drift out of alignment.

A settings panel collects the personal details that the rest of the app depends on, including the user's display name, their location and their followed sports teams. The display name is reused in the greeting and the AI assistant, the location drives local weather and local news, and the selected teams populate the sports feed. This gives the project a controlled, personalised entry point where later features draw on a shared set of preferences rather than asking for the same information repeatedly.

4.2 Home

The Home screen provides a summary of the user's day. It greets the user by name and shows the current weather and a multi-day forecast for their location, a timeline of upcoming calendar events, their next highlighted event or work shift, and a launch strip of their chosen applications. Weather is retrieved through the weather hook, which calls the backend weather service and in turn the OpenWeather provider, returning the current conditions, a short human-readable summary and the forecast. The upcoming events are drawn from the user's connected calendars. The layout is designed so that all of this fits within a single screen without scrolling, acting as a launch point into the rest of the app.

4.3 Launchpad

The Launchpad lets the user open real applications on their machine directly from the dashboard. The backend inspects the system's application directories to list installed apps, extracts each application's own icon and serves it as an image, and opens an application on request. Because this feature works directly with the local operating system, it is designed to run when the backend is hosted on the user's own machine.

In version 2 the Launchpad became a view in its own right rather than a modal picker. It now shows the user's pinned applications alongside a separate collection of saved websites, and provides a single search field that searches installed applications, saved sites and the open web together. Web search is served by the backend's search service, which can use Brave or Tavily where a key is configured and otherwise falls back to a keyless open-web provider.

4.4 Calendar and Life Hub

Calendar functionality is handled through a dedicated calendar service that supports three sources: Google Calendar over OAuth, Apple iCloud through the CalDAV protocol, and subscribed public `.ics` feeds. A connection panel lets the user link or disconnect each provider, paste a public calendar feed, and toggle the visibility of individual calendars. Once connected, events sync into the app and feed both the home screen's upcoming timeline and the Life Hub schedule. The layered provider design means each calendar source is handled by its own provider behind a common service interface.

The Life Hub is the planning area of the application. It presents the day's schedule alongside a to-do list, a set of projects with their own nested tasks and completion progress, and a list of daily habits with a running completion count. The life data hook manages this state and its persistence, allowing tasks and habits to be created, checked off and cleared, with projects tracking how many of their tasks are complete.

4.5 Markets & News

The Markets & News area combines several real-time data sources into one screen. A live television news stream is played directly in the browser using `hls.js`, with channel controls, a link to the source site and a full-screen immersive mode. Between the hero and the ground, a continuously scrolling ticker shows live prices for a watchlist of stocks and cryptocurrencies, retrieved through the market data hook from the backend's Finnhub and CoinGecko providers. A configurable feed panel lets the user switch between news, sport and market views. News is gathered on the backend through the GNews provider supplemented by RSS feeds, with local news resolved from the user's town or county.

The sport feed supports football, the NBA, the NFL and Formula 1. Depending on the selected team or competition, it shows the next fixture and a full standings table, such as a Premier League table with the user's team highlighted, or the Formula 1 drivers' and constructors' championships with team badges and points. On the backend these are served by separate providers — Football-Data.org for football, `balldontlie` for the NBA and NFL, and the key-free Jolpica (Ergast) service for Formula 1 — each wrapped by its own service so the different data shapes are normalised before reaching the interface. The markets feed provides a watchlist showing each instrument's price and percentage change, with a summary of how many are up or down, sourced live from Finnhub.

4.6 Music

The Music area lets the user control playback on their device through Spotify. Using the Spotify Web API, the Web Playback SDK and Spotify Connect, the player hook manages authentication, transfers playback between devices, and exposes a now-playing view with track details, a progress bar and transport controls. The user can browse their playlists, see what they have recently played and search the full Spotify catalogue. All Spotify communication is proxied through the backend so that secret credentials never reach the client.

Version 2 adds a full-screen immersive mode built on the same spatial system as the rest of the application, so that the player is a sky, a horizon and a ground like every other view. Here the horizon is the track itself: the rule spans the screen, the portion already played is lit, and the whole rule is a seek control, so any point in the track can be reached directly. The current position is shown as a light sitting within the rule rather than as a marker standing away from it, using the same layered streak that travels the navigation baseline elsewhere in the interface; a second such light sweeps the played portion on a continuous loop, its travel bounded by the playhead so that it never crosses part of the track that has not yet been reached. Lyrics are fetched from the keyless LRCLIB service, synchronised to playback position, and set in the display typeface above the line; the line currently being sung is marked with the same accent tick used to head every column elsewhere in the interface. Below the line the album sleeve is presented as a physical object alongside the transport controls.

The colour of the whole screen is sampled from the album artwork and nothing else; the atmosphere deliberately does not react to the audio, since the screen is intended to be left running rather than watched. If the machine is left idle while something is playing, this immersive player takes over as the screensaver in place of the idle clock, hiding the transport and giving the ground to the clock.

4.7 Travel

In version 1 the Travel planner was deliberately kept local to the device: a countdown to an upcoming trip, with flight and hotel details, a packing list and an itinerary all typed in by hand. Version 2 connects it to live data and it is now the largest functional change in the release.

A trip now shows real flight information — airline, route, scheduled times, distance and time in the air — retrieved through the backend's flights and schedule providers. The destination is drawn on a real vector map rendered with MapLibre GL over free OpenFreeMap tiles, showing the route, airports, saved places and accommodation. Alongside this the view presents live local time and weather for the destination, a currency converter working from live rates, and practical destination facts such as plug type, voltage, driving side, dialling code and emergency numbers. Places can be searched and added directly into a day-by-day itinerary. The packing checklist and trip state remain on the device.

4.8 The AI assistant

At the centre of the navigation sits the AI assistant, which is more than a simple chat feature. It runs an agent loop: the conversation is sent to the model together with a set of tool definitions and a system prompt, and while the model responds with tool calls, the app executes each tool, feeds the result back, and continues until the model returns a final answer, up to a fixed number of steps. This allows the assistant to read the user's live data — their weather, calendar, tasks, markets or music — and to make changes on their behalf, rather than only producing text. On the backend, the AI service resolves which provider to use, defaulting to Google Gemini but able to switch to OpenAI or Anthropic Claude per request, and each provider is wrapped so their different message and function-calling formats are handled behind a common interface.

Version 2 adds a real-time spoken interface to the same assistant. The browser opens a WebSocket to the Express server, which bridges that connection to the Gemini Live API; microphone audio is captured by an AudioWorklet and sent as 16 kHz PCM, and the model's speech returns at 24 kHz. Turn-taking, voice activity detection and interruption are handled server-side, so the user can speak over the assistant naturally. Crucially, the voice session reuses exactly the same tool definitions as the typed assistant and executes them against the same live application state, so speaking to Pulse and typing to it can do the same things. Double-tapping the space bar anywhere in the application starts a voice conversation.

4.9 Rate limiting and usage capping

The project includes a request rate limiting and usage capping system designed to keep the application safe to run and free to operate. Every API route passes through an in-memory rate limiter that caps requests per client to absorb runaway polling, with tighter limits on the AI route and on any request that changes state. Separately, a persisted usage meter enforces hard totals on the metered AI providers, checking a provider's daily and monthly request and token budgets before each call so a cap can be reached but never exceeded. The default Gemini budgets are deliberately set beneath the provider's free-tier limits, meaning the AI assistant can run without ever incurring a charge, and because the counters are written to disk they survive restarts rather than resetting. The server also boots with no API keys at all: any integration that is not configured returns a clear "not configured" response rather than crashing, and a status endpoint reports exactly which integrations are live.

4.10 Removed in version 2

The Finance overview present in version 1 — liquid balance, net worth, income and expenses, a monthly spending trend, accounts, upcoming bills and savings goals — was removed. It was held entirely in local storage and populated by hand, and unlike every other domain in the application there was no responsible way to make it live without asking a user to supply banking credentials to a personal project. Rather than retain a view that only worked convincingly as a demonstration, it was cut from the product.